

NLP

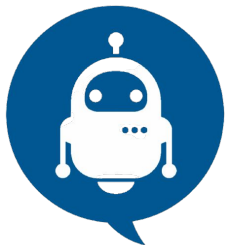
Sequence to Sequence (seq2seq)

MSc. Fernando Silva
fernandosilva.clases@gmail.com

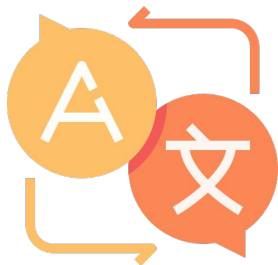
Soluciones Seq2Seq



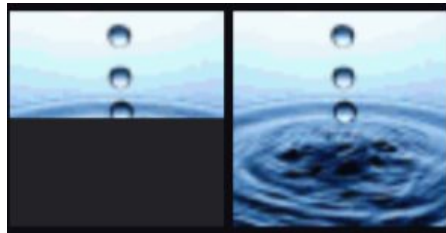
Trabaja principalmente con el concepto many-to-many en formato "codificador" a "decodificador", en donde la secuencia de entrada se codifica a una representación intermedia y se decodifica al espacio de salida. Es útil para traducir o transferir representaciones entre distintos dominios de datos o modalidades.



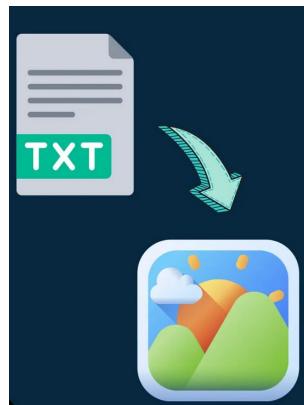
Bots Conversacionales
(modelos de lenguaje)



Traducción de
idiomas



Completar una
imagen



Text2img

Seq2Seq



Pensemos en la tarea de **traducción de idiomas**:

Hola mi nombre es Juan Perez

Hi my name is Juan Perez

$x_1, x_2, \dots, x_m$



$y_1, y_2, \dots, y_n$

(Las longitudes pueden ser diferentes)

La traducción puede entenderse como la búsqueda de la secuencia objetivo que sea la más probable dada la entrada. Aprendemos una función $p(y|x, \theta)$ con algunos parámetros θ luego encontramos su argmax para una entrada dada:

$$y' = \arg \max_y p(y|x, \theta)$$

Parámetros

Modelo

Seq2Seq, Conditional Language Models



- ❖ En los **Language Models** se estima la **probabilidad incondicional** de una secuencia completa, usualmente factorizada de manera autoregresiva.
- ❖ En cambio, los modelos **Seq2Seq** pueden verse como **Conditional Language Models**, donde se **estima la probabilidad condicional** $p(y|x)$, es decir, la probabilidad de una secuencia objetivo dada una secuencia fuente.

Language Models

Un LM asigna probabilidades a secuencias de palabras, $w = (w_1, w_2, \dots, w_n)$. Ej: (“Hola”, “mi”, “nombre”, “es”, “Juan”, “Perez”).

$$p(w) = p(w_1) \times p(w_2 | w_1) \times p(w_3 | w_1, w_2) \times \dots \times p(w_\ell | w_1, \dots, w_{\ell-1})$$
$$= \prod_{t=1}^{|w|} p(w_t | w_1, \dots, w_{t-1})$$

Conditional Language Models

Un Conditional LM asigna probabilidades a secuencias de palabras, $w = (w_1, w_2, \dots, w_n)$ dado un contexto de condicionamiento, x .

$$p(w | x) = \prod_{t=1}^{\ell} p(w_t | x, w_1, w_2, \dots, w_{t-1})$$

Seq2Seq, Conditional Language Models



Para entrenar modelos de lenguaje condicionales, necesitamos muestras emparejadas $\{(\mathbf{x}_i, \mathbf{w}_i)\}_{i=1}^N$

Dataset Machine Translation

```
Do we need a universal language?      Avons-nous besoin d'un langage universel ?      CC-BY 2.0 (France)
Attribution: tatoeba.org #471875 (blay_paul) & #473921 (sacredceltic)
Do we need a universal language?      A-t-on besoin d'une langue universelle ?      CC-BY 2.0 (France)
Attribution: tatoeba.org #471875 (blay_paul) & #2808482 (Rockaround)
Do you always get up before six?      Est-ce que tu te lèves toujours avant six heures ?      CC-BY 2.0 (France)
Attribution: tatoeba.org #3396573 (CK) & #7966990 (Aiji)
Do you believe in reincarnation?      Croyez-vous à la réincarnation ?      CC-BY 2.0 (France) Attribution:
tatoeba.org #2642489 (CK) & #6461267 (Aiji)
```

Do we need a universal language?	Avons-nous besoin d'un langage universel ?	CC-BY 2.0 (France) Attribution: tatoeba.org #471875 (blay_paul) & #473921 (sacredceltic)
----------------------------------	--	--

Fuente: <https://www.manythings.org/anki/>

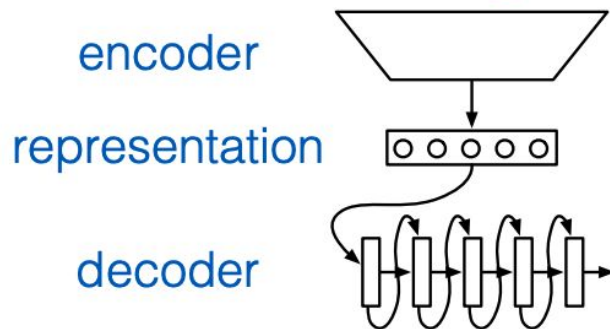
Seq2Seq, Conditional Language Models



Para entrenar modelos de lenguaje condicionales, necesitamos muestras emparejadas $\{(\mathbf{x}_i, \mathbf{w}_i)\}_{i=1}^N$

Dataset Machine Translation

$\mathbf{x}$ *Kunst kann nicht gelehrt werden...*



$\mathbf{w}$ *Artistry can't be taught...*

Seq2Seq, Conditional Language Models



Para entrenar modelos de lenguaje condicionales, necesitamos muestras emparejadas $\{(\mathbf{x}_i, \mathbf{w}_i)\}_{i=1}^N$

Dataset Image Captioning



Someone in a blue shirt and hat is standing on stair and leaning against a window.

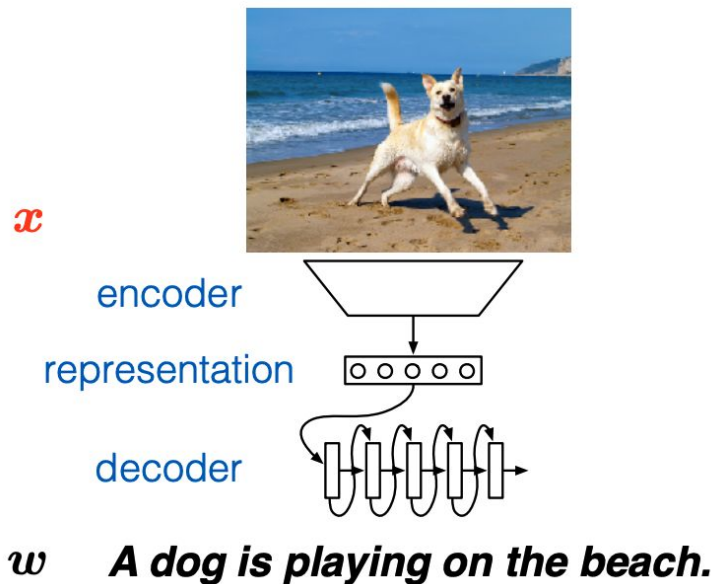
Fuente: <https://huggingface.co/datasets/nlphuji/flickr30k?image-viewer=01FFE5EA17DA0E65D937760CBF044A25995FFD63>

Seq2Seq, Conditional Language Models



Para entrenar modelos de lenguaje condicionales, necesitamos muestras emparejadas $\{(\mathbf{x}_i, \mathbf{w}_i)\}_{i=1}^N$

Dataset Image Captioning

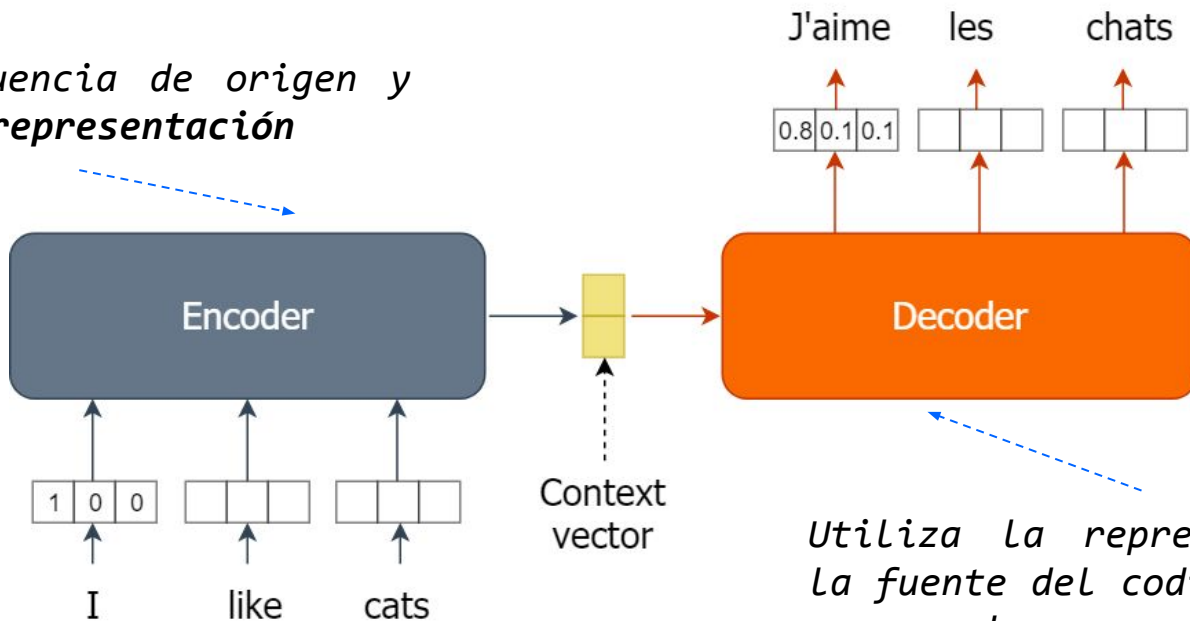


Seq2Seq, Arquitectura Encoder-Decoder



Esta orientado a tareas de secuencia a secuencia. Este marco consta de dos componentes: **Encoder** y **Decoder**.

*Lee la secuencia de origen y produce su **representación***

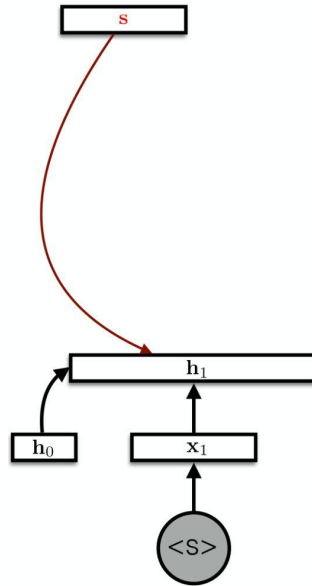


*Utiliza la representación de la fuente del codificador para **generar** la secuencia objetivo.*

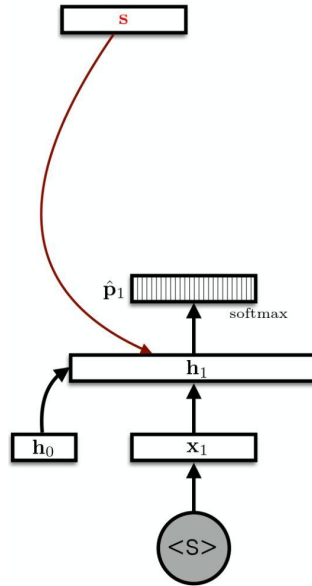
Seq2Seq, Arquitectura Encoder-Decoder



El encoder procesa toda la secuencia de entrada produciendo un estado oculto que se pasará como **s** al decoder



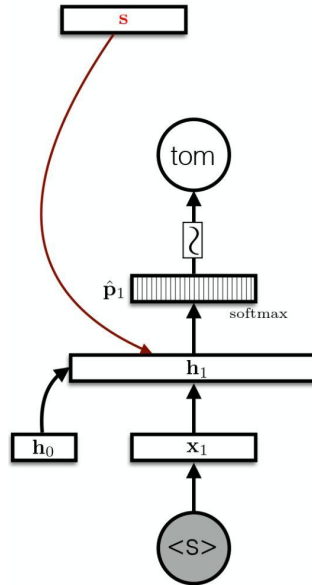
Seq2Seq, Arquitectura Encoder-Decoder



Seq2Seq, Arquitectura Encoder-Decoder



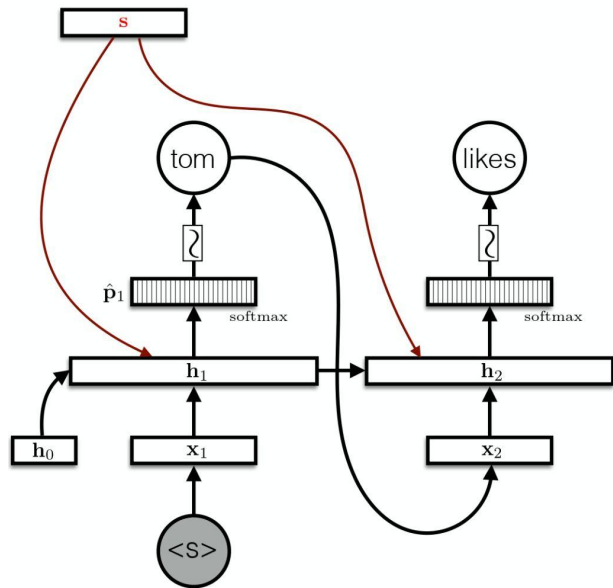
$$p(\text{tom} \mid \textcolor{red}{s}, \langle s \rangle)$$



Seq2Seq, Arquitectura Encoder-Decoder



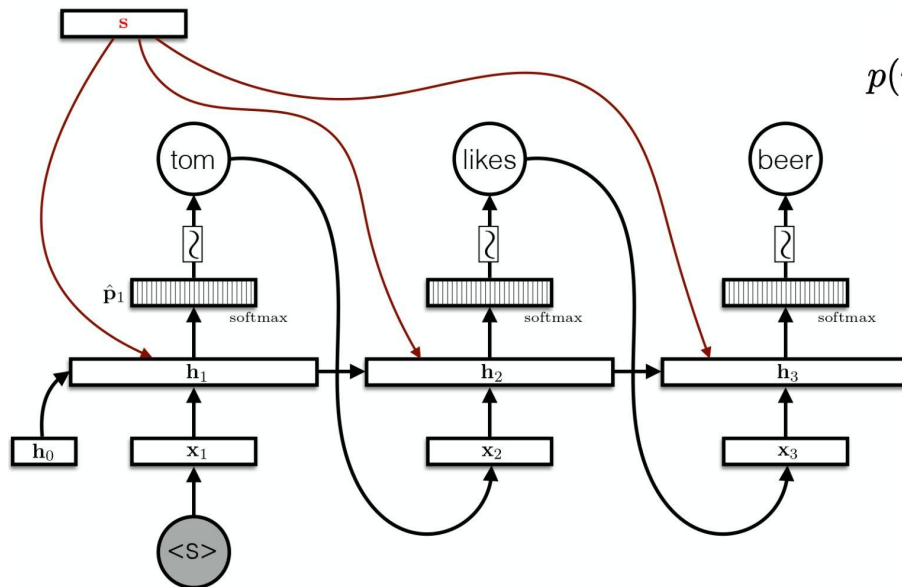
$$p(\text{tom} \mid \textcolor{red}{s}, \langle \text{s} \rangle) \times p(\text{likes} \mid \textcolor{red}{s}, \langle \text{s} \rangle, \text{tom})$$



Seq2Seq, Arquitectura Encoder-Decoder



$$p(\text{tom} \mid \mathbf{s}, \langle \mathbf{s} \rangle) \times p(\text{likes} \mid \mathbf{s}, \langle \mathbf{s} \rangle, \text{tom}) \\ \times p(\text{beer} \mid \mathbf{s}, \langle \mathbf{s} \rangle, \text{tom}, \text{likes})$$

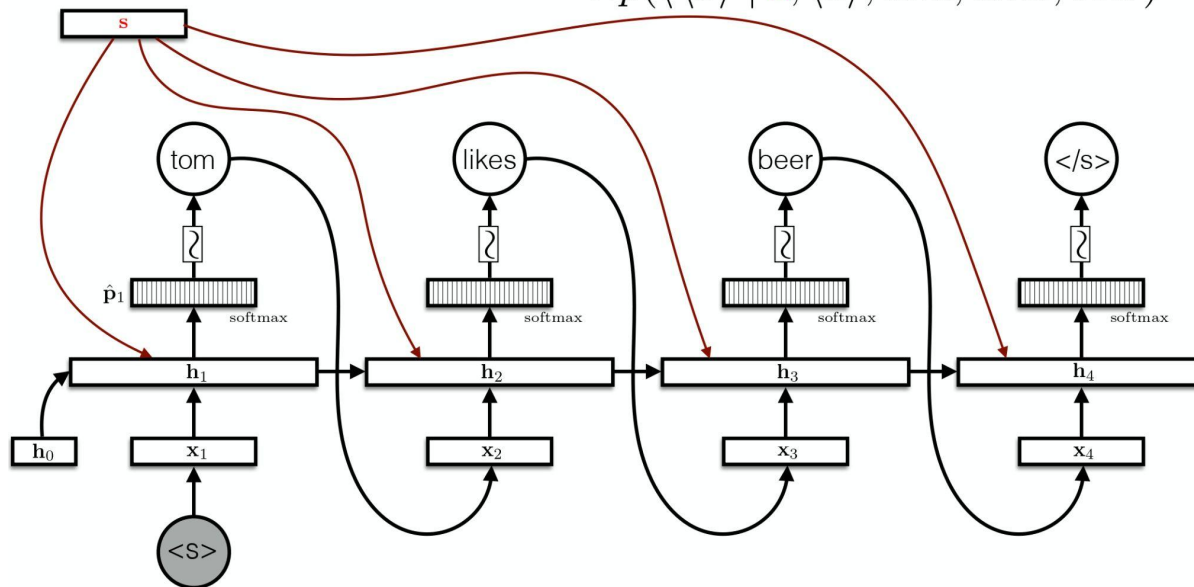


$$p(\mathbf{w} \mid \mathbf{x}) = \prod_{t=1}^{\ell} p(w_t \mid \mathbf{x}, w_1, w_2, \dots, w_{t-1})$$

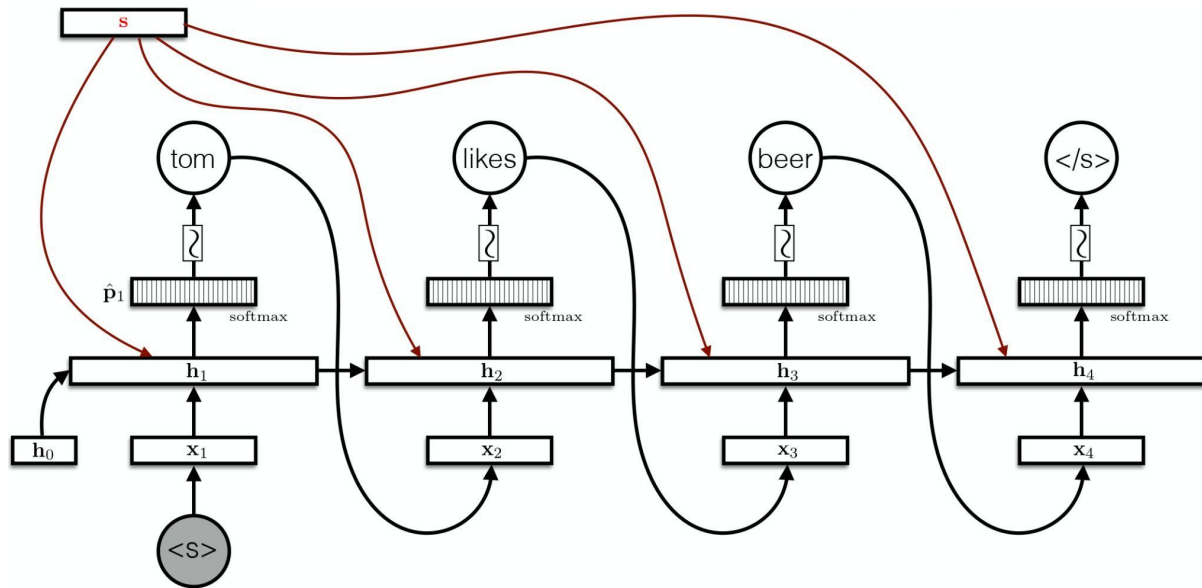
Seq2Seq, Arquitectura Encoder-Decoder



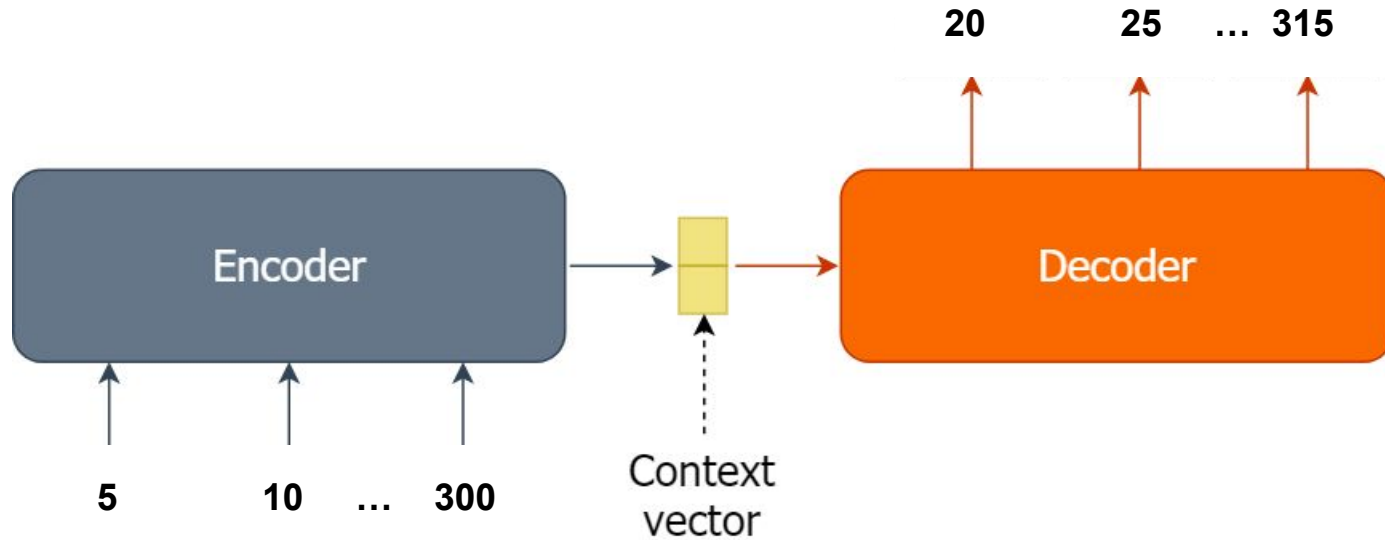
$$p(\text{tom} \mid \mathbf{s}, \langle \mathbf{s} \rangle) \times p(\text{likes} \mid \mathbf{s}, \langle \mathbf{s} \rangle, \text{tom}) \\ \times p(\text{beer} \mid \mathbf{s}, \langle \mathbf{s} \rangle, \text{tom}, \text{likes}) \\ \times p(\langle \backslash \mathbf{s} \rangle \mid \mathbf{s}, \langle \mathbf{s} \rangle, \text{tom}, \text{likes}, \text{beer})$$



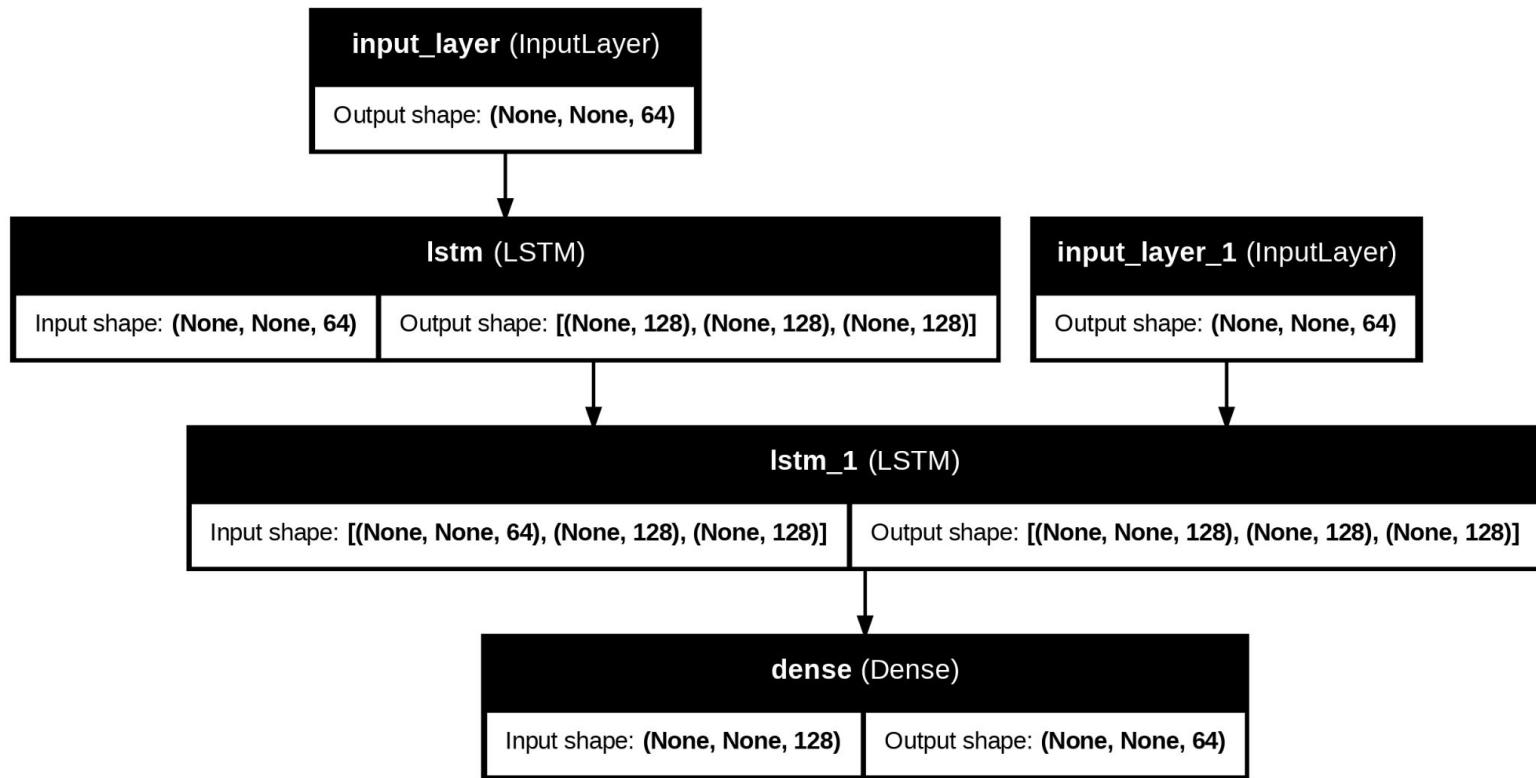
Seq2Seq, Arquitectura Encoder-Decoder



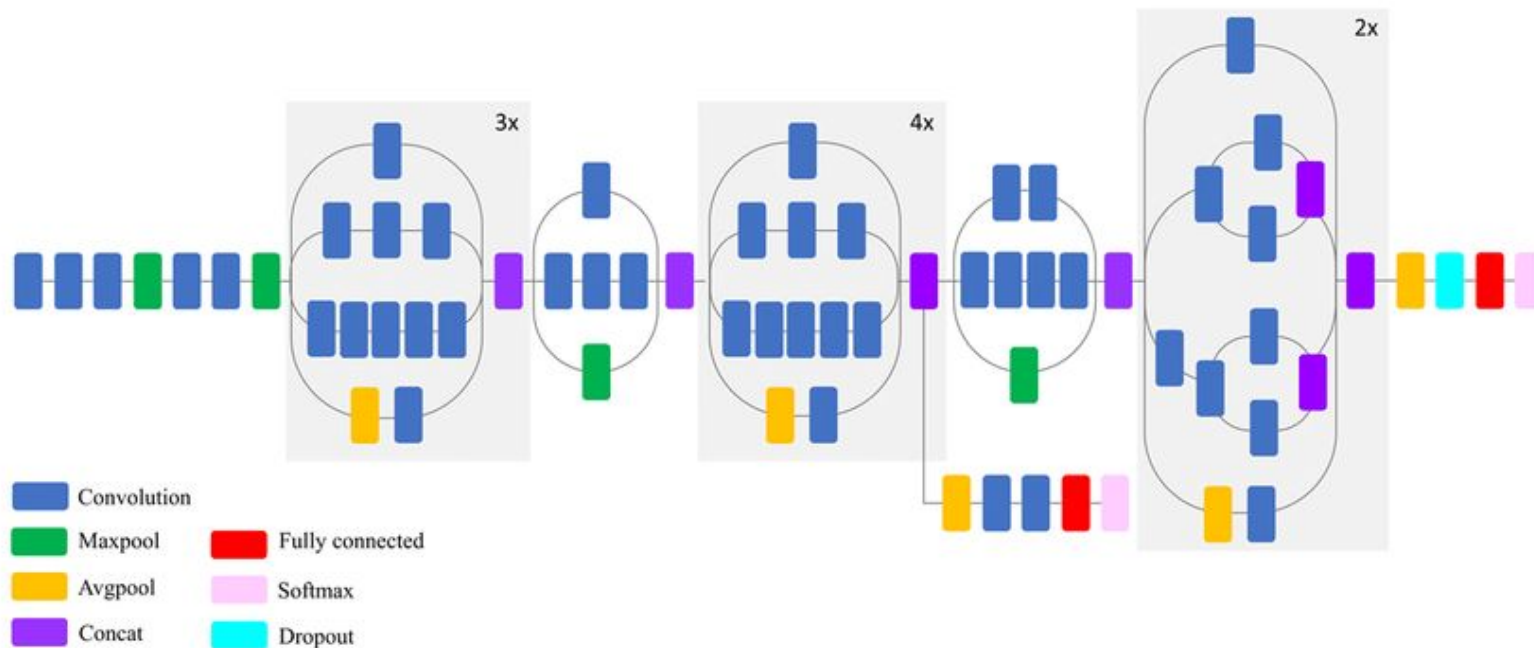
Ejercicio: Encoder-Decoder



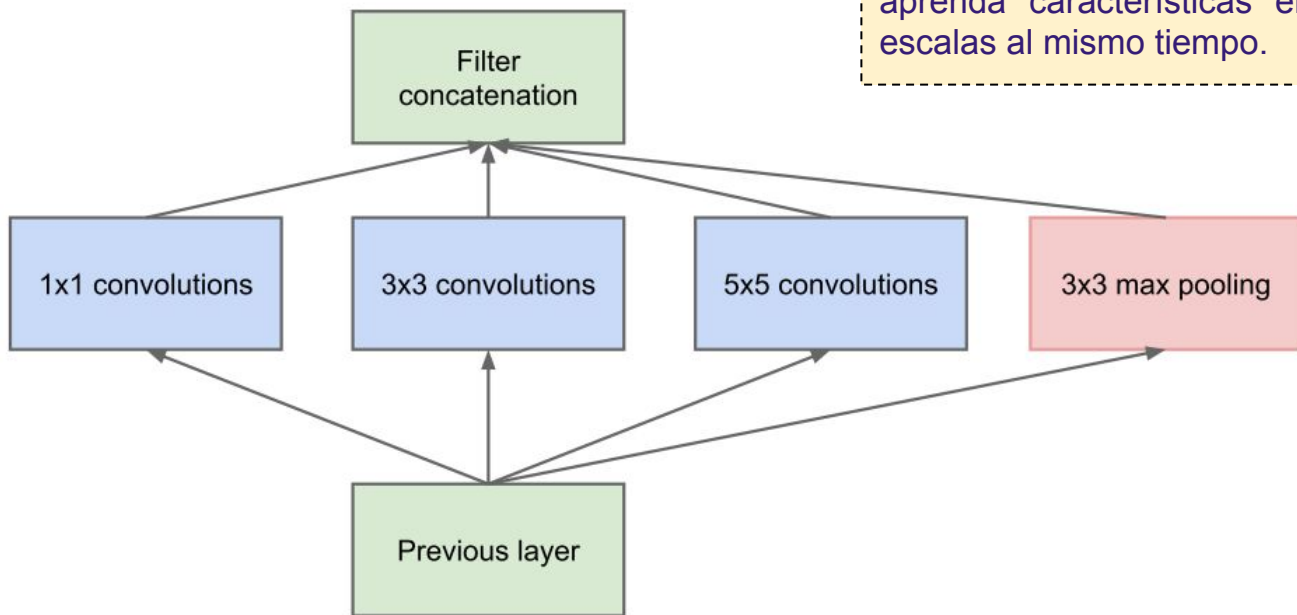
Ejercicio: Encoder-Decoder



Paréntesis: Functional en Keras



Paréntesis: Functional en Keras



Functional permite que una CNN aprenda características en múltiples escalas al mismo tiempo.

 <https://colab.research.google.com/drive/1Pio5Jo946el9wQ0pSqHJYiP8V0e2-kDG?usp=sharing>



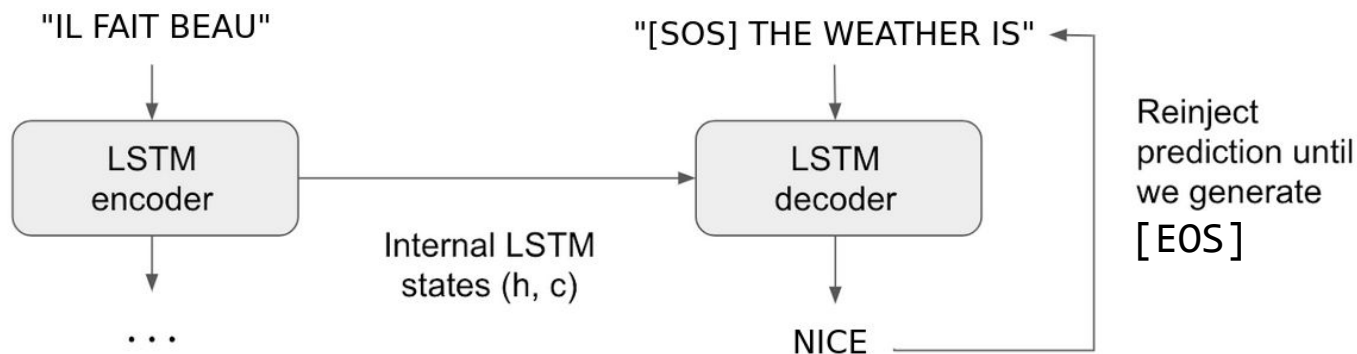
Link al Colab



LINK



Cuando hablamos de un encoder-decoder NLP se agrega un grado de dificultad más, ya que las secuencias no necesariamente tienen el mismo tamaño y que hay que vectorizar las sentencias de entrada

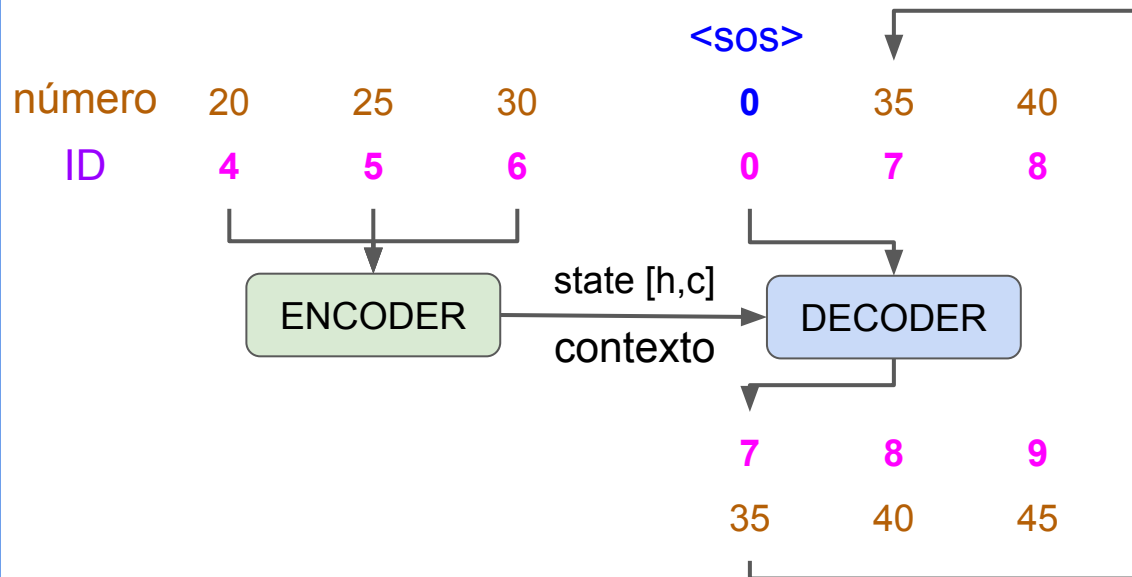


Para solucionar el problema de secuencias de distinto tamaño se define una máxima longitud y luego se acota con los tokens de inicio y fin de sentencia (<sos>/<eos>)

Tokens especiales <SOS> & <EOS>



Tokens que se reservan para indicarle al modelo el comienzo (Start Of Sequence) o el fin (End Of Sequence) de la secuencia.



Las **palabras/números** se transforman en tokens (**ids**) con el Tokenizador o LabelEncoder

En este ejemplo de secuencia numérica usaremos el número “0” como **<sos>**.

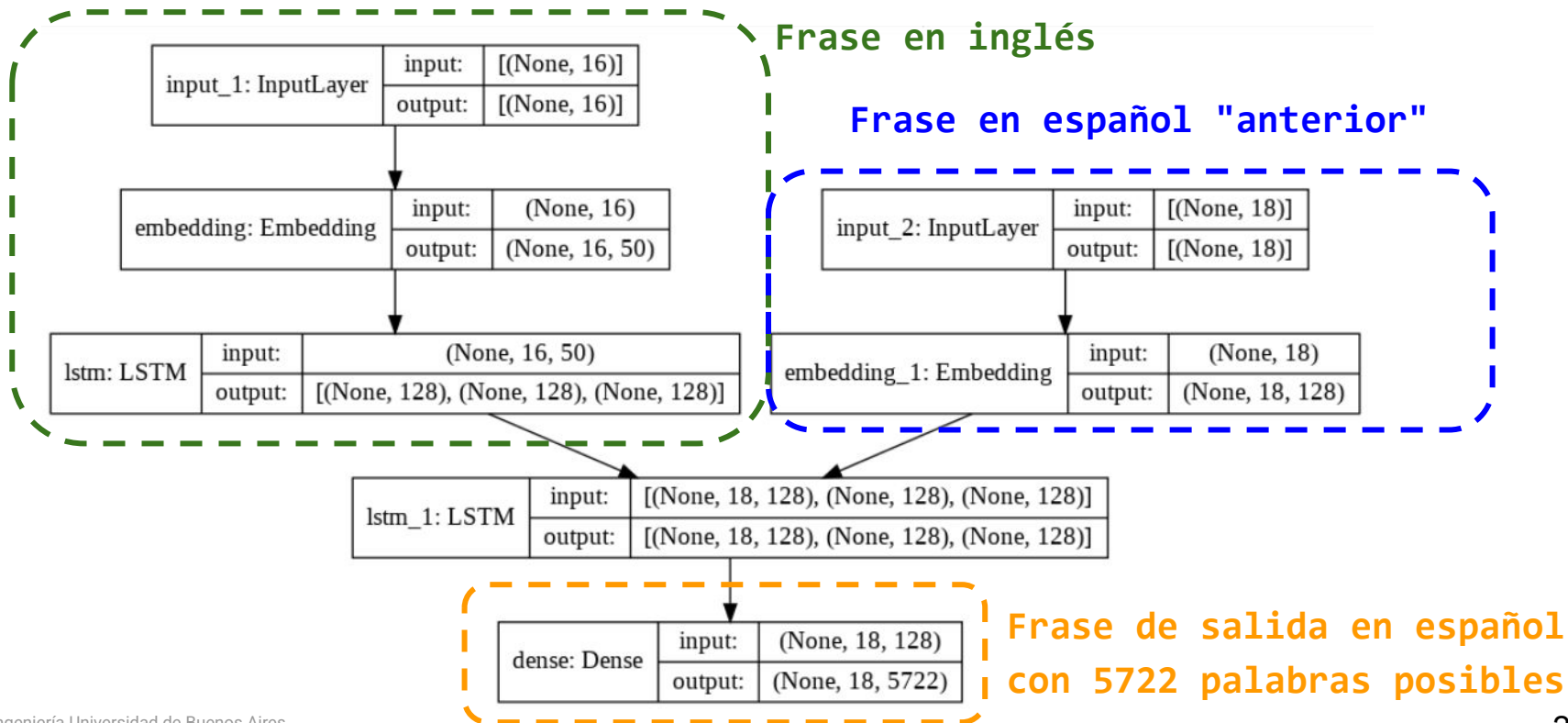
El LabelEncoder le dará un **ID** a ese token especial (en este caso también 0)

Es importante que el token esté representado por una palabra/número que no exista en el vocabulario (para no confundirlo)

Traductores



En este ejemplo realizaremos un traductor de inglés a español, vectorizando las sentencias de entrada con Embeddings



Inferencia del traductor



El encoder inicializa el contexto (h1,c1) con la entrada del decoder en <sos>, luego la salida es realimentada.

```
('A deal is a deal.',  
'Un trato es un trato. <eos>',  
'<sos> Un trato es un trato.')
```

Ejemplo
anterior:

```
input = [1, 2, 3]  
output = [4, 5, 6]  
output_in = [0, 4, 5]
```

```
Input: Tom is naked.  
Response: tom es un noche
```

Ensayo real, formó una
oración coherente pero no era
el resultado solicitado

Step 1:

A deal is a deal -> Encoder -> enc(h1,c1)

enc(h1,c1) + <sos> -> Decoder -> Un + dec(h1,c1)

step 2:

dec(h1,c1) + Un -> Decoder -> trato + dec(h2,c2)

step 3:

dec(h2,c2) + trato -> Decoder -> es + dec(h3,c3)

step 4:

dec(h3,c3) + es -> Decoder -> un + dec(h4,c4)

step 5:

dec(h4,c4) + un -> Decoder -> trato + dec(h5,c5)

step 6:

dec(h5,c5) + trato. -> Decoder -> <eos> + dec(h6,c6)



Link al Colab



[LINK](#)

Desafío 4 (traductor)



Replicar y extender el traductor:

- Extender el entrenamiento a más datos y tamaños de secuencias mayores.
- Explorar el impacto de la cantidad de neuronas en las capas recurrentes.
- Mostrar 5 ejemplos de traducciones generadas.

Opcional:

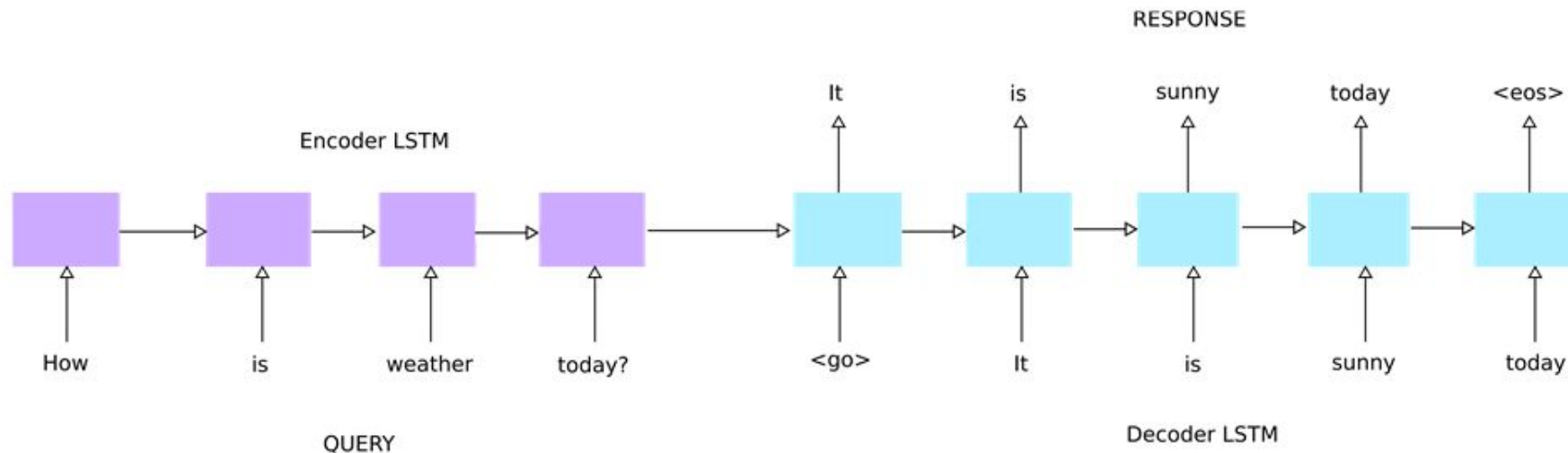
- Embeddings pre-entrenados para los dos idiomas; cambiar la estrategia de generación (por ejemplo muestreo aleatorio).
- Replicar el modelo en PyTorch.

Question and answers (QA) ~ Bot

[LINK](#)



Es hora de armar un Bot conversacional, que responda a preguntas que nosotros le hagamos (QA). Para ello utilizaremos un dataset "modesto" por lo que no se espera alcanzar resultados muy prometedores



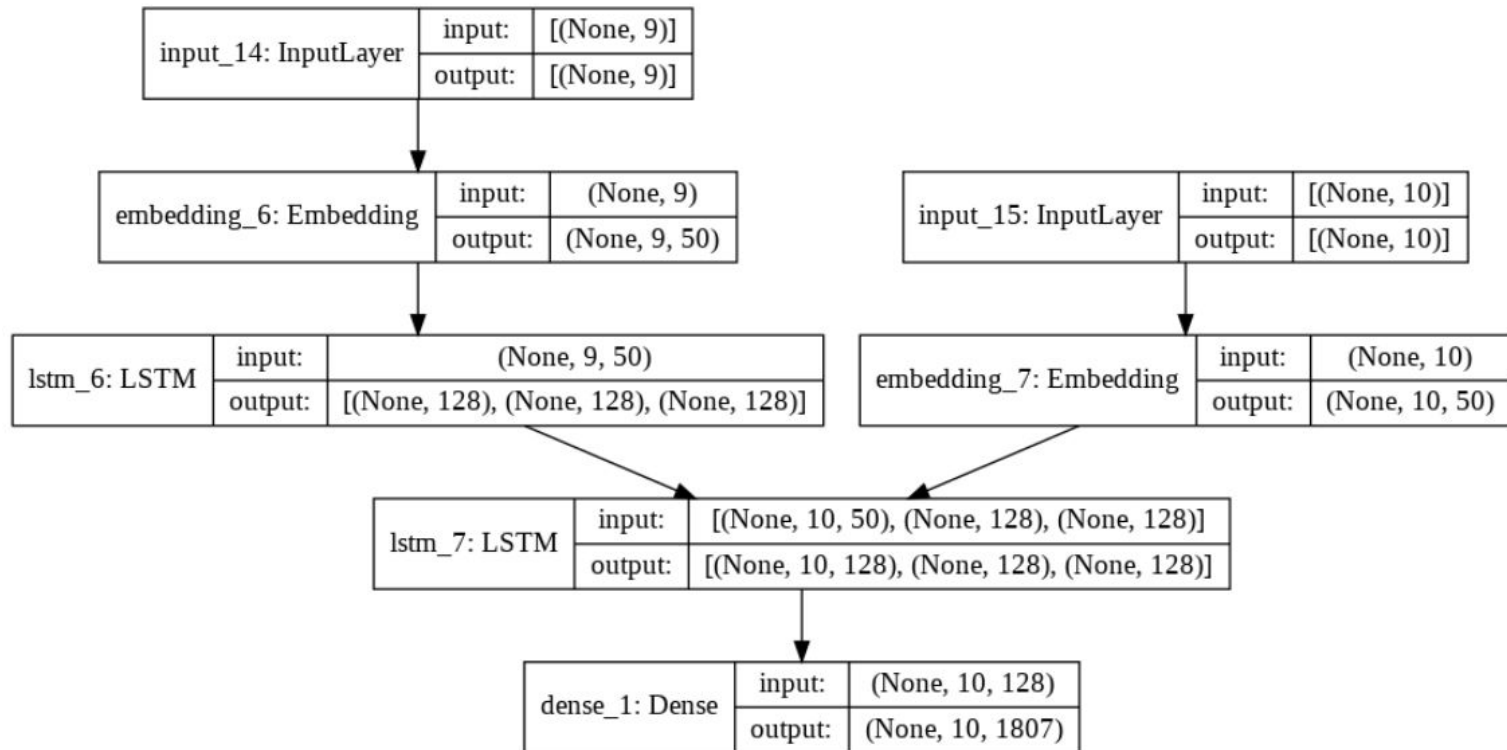
Input: i like to go to the beach
Response: i am a vegan

Question and answers (QA) Bot

[LINK](#)



Un ejemplo de una arquitectura utilizando los Embeddings de Glove (dim 50)





Construir QA Bot basado en el ejemplo del traductor pero con un dataset QA

[LINK](#)

Recomendaciones para evitar esto...



Recomendaciones:

MAX_VOCAB_SIZE = 8000
max_length ~ 10
Embeddings 300 Fasttext
n_units = 128
LSTM Dropout 0.2
Epochs 30~50

Preguntas interesantes:

Do you read?
Do you have any pet?
Where are you from?